

IMX296 Register Reference

Derived from `nv_imx296.c` and `imx296_mode_tbls.h`

Sony IMX296LQ/LL image sensor driver for NVIDIA Jetson (tegracam framework)

1 Driver status

Snapshot of what the driver actually does today, kept in sync with `nv_imx296.c` / `imx296_mode_tbls.h`. See the project `README.md` for the full known-issues list; only the items relevant to register behavior are called out here.

- **One mode only:** 1456x1088 full pixel array, 60 fps max. No cropped or binned mode is currently wired up, even though the sensor's `WINMODE` / `ROI` registers support it (see Section 10).
- **Debug dump left on:** `nv_imx296.c` unconditionally `#defines` `DEBUG`, so the full register dump in `imx296_dump_registers()` (documented throughout this reference) runs on every `set_mode/` `start_streaming/` `stop_streaming`, adding real latency. Harmless for correctness, but not representative of a production build.
- No HDR, EEPROM, or OTP support; `TEGRA_CAMERA_CID_*` list is deliberately minimal (gain, exposure, frame rate, fuse ID).
- Monochrome (IMX296LL) variant is auto-detected in code but has not been hardware-tested; only the color IMX296LQ has been run.

2 Status legend

Known	Register's address, bit layout, and value meaning are stated in the driver source (from the mainline Linux driver, libcamera, or direct comments).
Partial	Register is read/dumped or written with a fixed value, and a plausible meaning is documented, but the bit-level semantics are not confirmed by any cited source (marked "should be" / inferred).
Unknown	Register is written (or read) with a value copied from vendor/mainline tables, but the driver comments explicitly say the meaning is undocumented – address and value are known, function is not.

3 Streaming / mode control

Addr	Name	Values	Description	Status
0x3000	CTRL00	0x00 = operating 0x01 = standby (IMX296_CTRL00_STANDBY, bit 0)	Master standby control. Must be 0x00 (operating) to read <code>SENSOR_INFO</code> (0x3148). Sensor enters standby after <code>stop_streaming()</code> .	Known
0x300a	CTRL0A	0x00 = start MIPI transmission 0x01 = stop (IMX296_CTRL0A_XMSTA, bit 0)	MIPI transmission start/stop. Cleared 2–300ms after <code>CTRL00</code> is cleared in <code>imx296_start_streaming()</code> ; set before <code>CTRL00=standby</code> in <code>imx296_stop_streaming()</code> .	Known
0x3008	CTRL08	bit 0 <code>REGHOLD</code> : 1 = hold writes, 0 = commit atomically at next frame	Group-hold register. Driver toggles this from <code>imx296_set_group_hold()</code> .	Known
0x300d	CTRL0D	bits[1:0] <code>WINMODE</code> : 00 = full array, 10 = full array + 2x2 binning (FD)	Window mode / binning control. Driver always writes 0x00 (full	Partial

Addr	Name	Values	Description	Status
		binning) bit[5] HADD_ON_BINNING bit[6] SAT_CNT	array, no binning) for both defined modes.	
0x300e	CTRL0E	bit 0 VREVERSE (vertical flip) bit 1 HREVERSE (horizontal mirror)	Flip/mirror, applied via read-modify-write in <code>imx296_start_streaming()</code> from the <code>vertical-flip / horizontal-mirror</code> DT properties.	Known
0x300b	CTRL0B	dump comment: bit 0 = “trigger enable”; observed default 0x00 = free run	Read in <code>imx296_dump_registers()</code> only – never written by this driver. Meaning taken from register name/position, not confirmed.	Partial
0x3021	WDSEL	dump comment: 0x00 = normal	Read-only in dump. Likely WDR/HDR mode select given the name and IMX296’s dual-gain design, but never written (driver disables HDR entirely via <code>ctrl_cid_list</code>).	Unknown
0x3024	SST	dump comment: bit 0 = “SST enable”	Read-only in dump; not written. “SST” meaning not stated anywhere in source.	Unknown
0x3026	CTRLTOUT	–	Read-only in dump, no value comment at all – purpose unknown.	Unknown
0x3029	CTRLTRIG	–	Read-only in dump, no value comment at all – purpose unknown.	Unknown

4 Clock / PLL configuration

Addr	Name	Values	Description	Status
0x3005	CLK_CTRL	0xf0	Written unconditionally in <code>imx296_common_regs</code> . Comment: “required – activates CSI-2 output (undocumented)”. Address/value known, mechanism not.	Partial
0x3089	INCKSEL0	37.125 MHz: 0x80 54 MHz: 0xb0 74.25 MHz: 0x80	Input clock select, byte 0 of 4. Value pinned to the 54 MHz path (Raspberry Pi GS camera on-board oscillator) in the current common-regs table.	Partial
0x308a	INCKSEL1	37.125 MHz: 0x0b 54 MHz: 0x0f 74.25 MHz: 0x0f	Input clock select, byte 1 of 4.	Partial
0x308b	INCKSEL2	37.125 MHz: 0x80 54 MHz: 0xb0 74.25 MHz: 0x80	Input clock select, byte 2 of 4.	Partial

Addr	Name	Values	Description	Status
0x308c	INCKSEL3	37.125 MHz: 0x08 54 MHz: 0x0c 74.25 MHz: 0x0c	Input clock select, byte 3 of 4.	Partial
0x418c	CTRL418C	37.125 MHz: 0x74 (116) 54 MHz: 0xa8 (168) – currently used 74.25 MHz: 0xe8 (232)	Clock-dependent register, exact function undocumented; only known via the three MCLK-indexed magic constants.	Unknown

Note: mode0/mode1 device-tree entries advertise mclk_khz = "54000", matching the 54 MHz INCKSEL/CTRL418C values actually written.

5 Timing: frame length (VMAX) and line length (HMAX)

Addr	Name	Values	Description	Status
0x3010	VMAX[7:0]	LSB of 17-bit (spec) / 24-bit (register space) frame length	Default 0x5e -> VMAX = 1118 lines = 1088 active + 30 blanking.	Known
0x3011	VMAX[15:8]	–	Default 0x04.	Known
0x3012	VMAX[22:16]	only bits[2:0] used, max value 0x1FFFF (17-bit)	Default 0x00. Overwritten at runtime by <code>imx296_set_frame_rate()</code> .	Known
0x3014	HMAX[7:0]	Fixed 0x4c	Line length = 1100 (74.25 MHz clock units) -> 14.81us/line. Never changed after init – no framerate control depends on HMAX in this driver.	Known
0x3015	HMAX[15:8]	Fixed 0x04	Combined with 0x3014: 0x044c = 1100.	Known

HMAX is fixed at init time and never revisited: the driver exposes a single mode, so there is no per-mode line-length switch to get wrong.

6 Exposure (SHS1)

Addr	Name	Values	Description	Status
0x308d	SHS1[7:0]	24-bit little-endian	Inverted shutter register: <code>exposure_lines = VMAX - SHS1</code> . Default 0x04 (SHS1=4 -> near-max exposure of 1114 lines 16.5ms).	Known
0x308e	SHS1[15:8]	–	Default 0x00.	Known
0x308f	SHS1[22:16]	only bits[2:0] used	Default 0x00. Overwritten at runtime by <code>imx296_set_exposure()</code> , clamped to <code>[1, frame_length - 4]</code> .	Known

`imx296_set_exposure()` converts the requested exposure time to coarse integration lines via `IMX296_EXPOSURE_OFFSET_US` and the mode's pixel clock / line length, per the libcamera `cam_helper_imx296.cpp` formula cited in the source comments, then derives SHS1 as `frame_length - coarse_time`.

7 Gain

Addr	Name	Values	Description	Status
0x3204	GAIN[7:0]	16-bit little-endian, 0–239	Gain in dB*10. 0 = 0dB (1.0x), 239 = 23.9dB (15.8x). Sensor converts dB to linear internally: $\text{linear} = 10^{(\text{reg}/200)}$. Overwritten at runtime by <code>imx296_set_gain()</code> .	Known
0x3205	GAIN[15:8]	–	Default 0x00.	Known
0x3200	GAINCTRL	0x01 = normal single-exposure gain (used) 0x41 = multi-exposure WDR gain (unused, no HDR)	Gain control mode.	Known
0x3212	GAINDLY	0x08 = no delay (used) 0x09 = 1-frame delay	Gain application delay. libcamera uses a 2-frame gain delay but this driver applies gain immediately and lets the tegracam framework handle delay compensation.	Known

8 Black level

Addr	Name	Values	Description	Status
0x3254	BLKLEVEL[7:0]	Default 0x3c = 60; 0x00 during test pattern	Black level target, 16-bit little-endian.	Known
0x3255	BLKLEVEL[15:8]	Default 0x00	–	Known
0x3022	BLKLEVELAUTO	0x01 = auto calibration on (used) 0xf0 = off, fixed BLKLEVEL (used during test pattern)	Automatic black-level calibration enable.	Known

9 Sync / master-slave mode

Addr	Name	Values	Description	Status
0x3036	SYNCSEL	0xc0 = normal/master: XVS/XHS output enabled (used) 0xf0 = HiZ/slave: XVS/XHS high-impedance	Sync output control. Driver uses master mode (sensor free-running); DT comment notes XVS/XHS pins are exposed for possible future multi-camera sync.	Known

10 ROI / windowing

The driver only ever defines the single full-array 1456x1088 mode (see Section 1); ROI is never enabled. The `FID0_ROIPH1/FID0_ROIWH1` position/width registers exist in the sensor and were used by an earlier, now-removed 1440x1088 cropped mode, but are not touched by the current `imx296_mode_tbls.h` – they are omitted from the table below since this reference only documents what the driver actually writes.

Addr	Name	Values	Description	Status
0x3300	FID0_ROI	bit 0 R0IH10N, bit 1 R0IV10N 0x00 = no ROI (only value ever written)	ROI window enable flags. Always written 0x00 – full array, no ROI.	Known

11 Test pattern generator

Addr	Name	Values	Description	Status
0x3238	PGCTRL	bit 0 REGEN pattern-gen enable bit 1 THRU bypass sensor data with pattern bit 2 CLKEN pattern clock enable bits[5:3] MODE: 0 Multiple pixels, 1 Sequence 1, 2 Sequence 2, 3 Gradient (used), 4 Row, 5 Column, 6 Cross, 7 Stripe, 8 Checks	Test pattern control. Driver enables gradient pattern (0x1d = REGEN CLKEN MODE(3)) when the test_mode module parameter is set.	Known
0x3239	PGHPOS[7:0]	0x08	Pattern horizontal start position = 8.	Known
0x323a	PGHPOS[15:8]	0x00	–	Known
0x323b	PGVPOS[7:0]	0x08	Pattern vertical start position = 8.	Known
0x323c	PGVPOS[15:8]	0x00	–	Known
0x323e	PGHPSTEP	0x08	Horizontal step size = 8.	Known
0x323f	PGVPSTEP	0x08	Vertical step size = 8.	Known
0x3240	PGHPNUM	0x64 = 100	Horizontal repeat count.	Known
0x3241	PGVPNUM	0x64 = 100	Vertical repeat count.	Known
0x3244	PGDATA1[7:0]	0x00	Combined with 0x3245: pattern data value 1 = 0x0300.	Known
0x3245	PGDATA1[15:8]	0x03	–	Known
0x3246	PGDATA2[7:0]	0x00	Combined with 0x3247: pattern data value 2 = 0x0100.	Known
0x3247	PGDATA2[15:8]	0x01	–	Known
0x3249	PGHGSTEP	0x00	Horizontal gray step = 0.	Known

12 Sensor identification / fuse ID

Addr	Name	Values	Description	Status
0x3148	SENSOR_INFO[7:0]	16-bit little-endian, split as: bit[15] = 1 mono (IMX296LL) / 0 color (IMX296LQ)	Chip ID / variant detect. Cannot be read while the sensor is in standby – driver exits standby, reads, then re-enters standby around this read.	Known

Addr	Name	Values	Description	Status
		bits[14:6] model number (expected 296)	Also reused as the 2-byte fuse/unique ID.	
0x3149	SENSOR_INFO[15:8]	–	High byte, see 0x3148.	Known

13 Undocumented vendor init sequence (imx296_common_regs)

The following registers are written once at `set_mode()` time from a block the source comments explicitly as “**extracted from vendor data that is entirely undocumented**”, copied verbatim from the mainline `imx296_init_table[]` / Sony reference design. Address and fixed value are known; register function is not documented anywhere in the driver, mainline driver, or libcamera. Every entry in this table is status Unknown.

Addr	Val	Addr	Val	Addr	Val
0x309e	0x04	0x31d0	0xf4	0x3833	0x00
0x30a0	0x04	0x321a	0x00	0x38a2	0xf6
0x30a1	0x3c	0x3226	0x02	0x38a3	0x00
0x30a4	0x5f	0x3256	0x01	0x3a00	0x80
0x30a8	0x91	0x3541	0x72	0x3d48	0xa3
0x30ac	0x28	0x3516	0x77	0x3d49	0x00
0x30af	0x09	0x350b	0x7f	0x3d4a	0x85
0x30df	0x00	0x3758	0xa3	0x3d4b	0x00
0x3165	0x00	0x3759	0x00	0x400e	0x58
0x3169	0x10	0x375a	0x85	0x4014	0x1c
0x316a	0x02	0x375b	0x00	0x4041	0x2a
0x31c8	0xf3	0x3832	0xf5	0x40a2	0x06
				0x40c1	0xf6
				0x40c7	0x0f
				0x40c8	0x00
				0x4174	0x00

0x31c8 and 0x31d0 carry an inline comment “exposure-related (undocumented)” – singled out from the rest of the block, but still without a stated bit meaning or unit.

14 Other named-but-unexplained registers

Addr	Name	Values	Description	Status
0x4114	GTTABLENUM	0xc5	Gamma table selection, per name. Comment only says “value from mainline driver” – no table contents or numbering scheme documented.	Partial

15 Summary counts

Known – address, bits, and meaning documented	37
Partial – address/value known, meaning inferred or partially stated	8
Unknown – address/value known, function undocumented	45

Counts are per register byte address, not per logical field (e.g. the 3-byte VMAX counts as 3). Roughly half of all registers this driver touches are copied from vendor/mainline tables with no documented function – treat any change to the

“Undocumented vendor init sequence” and “Other named-but- unexplained registers” sections as high-risk without hardware validation.